

DPO

Preferences without reinforcement learning · video 2:14

This lesson answers one question: can we learn from preferences without the whole RL machinery?

Direct Preference Optimization uses a mathematical shortcut: under a KL leash, the reward can be written in terms of the policy itself, so the model is its own reward model. Training becomes one loss over fixed preference pairs. On episode 38's task, DPO raises GPT-2's positivity from 0.14 to 1.08 in 51 steps (about two minutes), keeping fluent text; PPO reached 0.92 in about 19 minutes.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The model is its own reward model	▶ 0:08-0:54	2	☐
2. DPO on GPT-2	▶ 0:54-1:38	1	☐
3. PPO vs DPO	▶ 1:38-2:03	2	☐

Before you start: Deep Dive episodes 37 (reward models) and 38 (RLHF with PPO). Code: `code/d39_dpo` (GPT-2 small).

CONCEPT 1 OF 3

The model is its own reward model

▶ Video 0:08–0:54

RLHF with PPO maximises reward minus $\beta \times \text{KL}$ from a reference model. That objective has a known best policy: $p^*(y|x) \propto p_{\text{ref}}(y|x) \cdot \exp(\text{reward}(x, y) / \beta)$. Turned around, the reward can be written through the policy: **reward(x, y) = $\beta \cdot \log(p(y|x) / p_{\text{ref}}(y|x))$** + a term that depends only on the prompt x.

the best policy under a KL leash has a closed form

$$\text{reward}(x, y) = \beta \cdot \log(p_{\text{model}}(y | x) / p_{\text{ref}}(y | x))$$

(plus a term that depends only on the prompt x)

the model is its own reward model

▶ 0:38 Under a KL leash, $\text{reward}(x, y) = \beta \cdot \log(p_{\text{model}}(y|x) / p_{\text{ref}}(y|x))$ plus a term that depends only on x.

Plug that into the reward model's loss from episode 37, $-\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$: the prompt term cancels, and you get the **DPO loss** $-\log \sigma(\beta \cdot [\Delta_{\text{chosen}} - \Delta_{\text{rejected}}])$ with $\Delta = \log p(y|x) - \log p_{\text{ref}}(y|x)$. It raises the chosen answer's likelihood relative to the reference and lowers the rejected one's. No reward model, no value head, no sampling during training; the reference model is still needed.

episode 37: $\text{loss} = -\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$

DPO: $\text{loss} = -\log \sigma(\beta \cdot [\Delta_{\text{chosen}} - \Delta_{\text{rejected}}])$
 $\Delta = \log p_{\text{model}}(y | x) - \log p_{\text{ref}}(y | x)$

chosen: up, relative to the reference

rejected: down

no sampling during training

▶ 0:53 Substituting into the Bradley-Terry loss gives DPO.

KEY IDEA

DPO fits the policy so that its implicit reward explains the preferences.

Check 1: The model is its own reward model

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

$\beta = 0.1$, the chosen answer's log ratio Δ is +2 and the rejected one's -3 . What is the DPO loss for this pair? And when both Δ are 0 (the start of training)?

1.2 TRUE OR FALSE

"DPO needs no reference model."

True ☐ False ☐

CONCEPT 2 OF 3

DPO on GPT-2

▶ Video 0:54-1:38

The same goal as episode 38: positive continuations, scored by positive minus negative words. 1,536 continuations are sampled from GPT-2 once; every one that scored above zero is paired with a random lower-scoring one of the same prompt: 215 preference pairs. The original GPT-2 is the reference.

1,536 GPT-2 samples, scored once → 215 preference pairs

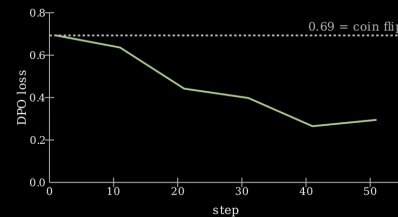
chosen (+1): "The movie was fully-functional and had good use for the reporter..."

rejected (0): "The movie was made 29 years ago and Spielberg fame is crazy now..."

every sample scoring above 0, paired with a lower-scoring one

▶ 1:08 1,536 samples scored once → 215 pairs (every sample scoring above 0 vs a lower one).

implicit reward prefers the chosen answer: 100% of the batch



▶ 1:21 DPO loss over 51 steps: 0.693 → under 0.3.

51 steps (3 epochs of batches of up to 16 pairs): the loss falls from 0.693 to under 0.3, and from the second epoch the implicit reward ranks the chosen answer higher in every logged batch. On fresh samples the reward rises from **0.14 to 1.08**, the text stays fluent ("Beautiful design with fantastic functions. And I love that..."), and the KL from GPT-2 is 3.8.

fresh samples, 8 prompts × 8

GPT2 0.14

after DPO 1.08

"My new phone is big. Beautiful design with fantastic functions.
And I love that it comes with an Infinity dot tag..."

KL from GPT-2: 3.8

▶ 1:36 Fresh samples: reward 0.14 → 1.08, fluent text, KL 3.8.

KEY IDEA

DPO's data is fixed in advance; all the learning signal is in the pairs.

Check 2: DPO on GPT-2

Answer all 1 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

Only 215 of 1,536 samples became pairs. Why so few?

CONCEPT 3 OF 3

PPO vs DPO

▶ Video 1:38-2:03

PPO at $\beta = 0.5$ reached 0.92 after about 19 minutes of sampling and scoring; DPO reached 1.08 in about two minutes, with no reward model, value head or sampling in the loop. (The KLs aren't measured identically: PPO's ≈ 6 comes from its last training batch, DPO's 3.8 from fresh samples.) That simplicity is a large part of why many open models are preference-tuned with DPO or its variants. PPO keeps one advantage: it learns from the model's own fresh samples, so it can keep improving beyond the fixed data.

	PPO (β 0.5)	DPO
reward	0.92	1.08
time	≈ 19 min	≈ 2 min
reward model in the loop	yes	no
value head	yes	no
sampling while training	yes	no

why many open models are tuned this way

▶ 1:53 PPO (β 0.5): 0.92, ≈ 19 min, reward model + value head + sampling. DPO: 1.08, ≈ 2 min.

KEY IDEA

Same objective, two routes: PPO optimises a learned reward online; DPO solves for the policy directly from offline pairs.

Check 3: PPO vs DPO

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

What does DPO need that ordinary supervised fine-tuning does not?

- A. A reward model
- B. A rejected answer for each example, and a reference model
- C. Sampling during training
- D. A value head

3.2 HANDS-ON CODE

Try it yourself. Compute DPO losses for a few pairs.

```
import torch, torch.nn.functional as F
beta = 0.1
d_c = torch.tensor([2.0, 0.0, -1.0]) # log p - log p_ref, chosen
d_r = torch.tensor([-3.0, 0.0, 1.0]) # same, rejected
print(-F.logsigmoid(beta * (d_c - d_r)))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The model is its own reward model · rewatch ▶ 0:08-0:54

1.1 0.474; 0.693. Margin $0.1 \times 5 = 0.5$, $-\log \sigma(0.5) = 0.474$; $-\log \sigma(0) = \ln 2$.

1.2 False. Δ is measured against the reference; it plays the role of PPO's KL anchor.

2. DPO on GPT-2 · rewatch ▶ 0:54-1:38

2.1 Most samples score 0 (no positive or negative words), and a pair needs a sample scoring above 0 plus a lower-scoring one. Plain GPT-2 averages only 0.14 positive words per continuation.

3. PPO vs DPO · rewatch ▶ 1:38-2:03

3.1 B. The loss compares chosen with rejected, both relative to the reference.

3.2 About [0.474, 0.693, 0.798]. Margins 0.5, 0, -0.2 .